МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра защиты информации



ЛАБОРАТОРНАЯ РАБОТА №3

по дисциплине: «Безопасность систем баз данных»

по теме: «Защита БД студий дубляжа»

Выполнил:

Студент гр. «АБс-322», «АВТФ»

Бутаков Никита Максимович

«___» _____ 2026г

_____

(подпись)

Проверил:

Ассистент кафедры ЗИ

Питько Яна Андреевна

«___» _____ 2026г

_____

(подпись)

Новосибирск 2026

Содержание

Введение 3
1. Шифрование паролей 4
2. Шифрование избранных столбцов 6
3. Реализация безопасного метода хранения ключей шифрования 9
4. Тесты производительности до и после включения шифрования 11
Вывод 13
Приложение А 14
Приложение Б 15
Приложение В 17
Приложение Г 26

Введение

Цели работы: изучить возможности шифрования в базе данных PostgreSQL и получить навыки работы с шифрованием БД на разных уровнях.

Тема: «Защита БД студий дубляжа – управление доступом к авторским материалам, контрактам с актёрами, расписаниям записей и защищённым архивам аудиофайлов».

Задачи:

1. Шифрование паролей;
2. Шифрование избранных столбцов;
3. Реализовать безопасный метод хранения ключей шифрования;
4. Провести тесты производительности до и после включения шифрования;
5. В выводе описать каждый метод шифрования БД.

1. Шифрование паролей

В системном каталоге pg_authid узнаем каким методом были зашифрованы пароли созданных пользователей (Рисунок 1).



```
     rolname      |             rolpassword
------------------+-------------------------------------
 app_owner_user   | md5c5be970772d7c11cbfd0e1beda9544ff
 security_auditor | md54ec2d1b40def557180fc7cbc27fb91fb
 studio_manager   | md5c8977d968b6da56eab849be8dec6ef79
 voice_actor      | md535a6698407ccb0469932ae1071ab1eae
(4 rows)
```

Рисунок 1 – Таблица pg_authid

Из рисунка 1 видим, что пароли хранятся в md5. Мы приняли решение установить метод хеширования на SCRAM-SHA-256.

SCRAM-SHA-256 использует случайную соль и многократные итерации, что обеспечивает защиту от перебора и радужных таблиц, а также поддерживает взаимную аутентификацию между клиентом и сервером и не допускает повторного использования сессии. К его недостаткам относится более высокие требования к ресурсам для хеширования и сложность реализации.

Алгоритм MD5 является простым и с минимальными ресурсами, он поддерживается практически любыми клиентскими библиотеками. Однако перехваченный хеш MD5 позволяет пройти аутентификацию без знания исходного пароля, и сам алгоритм уязвим к атакам по радужным таблицам.

Далее с помощью ALTER SYSTEM SET устанавливаем новый метод хранения паролей и перезаписываем те же пароли на новый хеш метод (Рисунок 2).

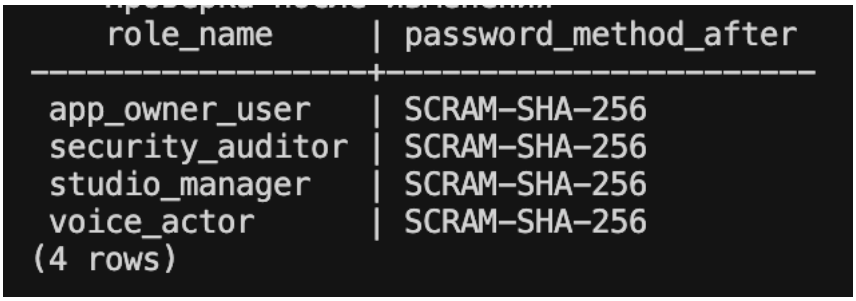


Рисунок 2 – Таблица pg_authid после изменения метода

Из рисунка 2 видим, что пароли перезаписались на новый метод хеширования. Код шифрования паролей представлен в приложении А.

2. Шифрование избранных столбцов

В нашей БД мы выбрали две таблицы: app.actors и ref.studios, в которых понадобится шифрование полей. Далее сгенерировали симметричный, открытый и закрытый PGP ключи с помощью команды gpg --full-generate-key.

После чего поле passport_data в таблице app.actors зашифруем с помощью симметричного ключа, а поле bank_details в таблице ref.studios зашифруем с помощью открытого ключа (Рисунок 3).

```
UPDATE app.actors
SET passport_data_enc = pgp_sym_encrypt(passport_data, current_setting('app.sym_key'))
WHERE passport_data IS NOT NULL AND passport_data_enc IS NULL;

UPDATE ref.studios
SET bank_details_enc = pgp_pub_encrypt(bank_details, dearmor(current_setting('app.public_key')))
WHERE bank_details IS NOT NULL AND bank_details_enc IS NULL;
```

Рисунок 3 – Шифрование полей

Теперь проверим, что данные без ключа выводятся в зашифрованном виде. На рисунке 4 представлена таблица app.actors с полем passport_data.

```
1  SELECT actor_id, encode(passport_data_enc, 'hex') AS passport_data_enc_hex
2  FROM app.actors
3  WHERE passport_data_enc IS NOT NULL
4  ORDER BY actor_id
```

Data Output | Messages | Notifications

Showing rows: 1 to 1

	actor_id [PK] integer	passport_data_enc_hex text
1	1	c30d0407030268f507978ecc469961d23d01a7d980b54e8bcc15721eb1c4f1804b37728c71c07d509

Рисунок 4 – Зашифрованные паспортные данные

На рисунке 5 представлена таблица ref.studios с полем bank_details.

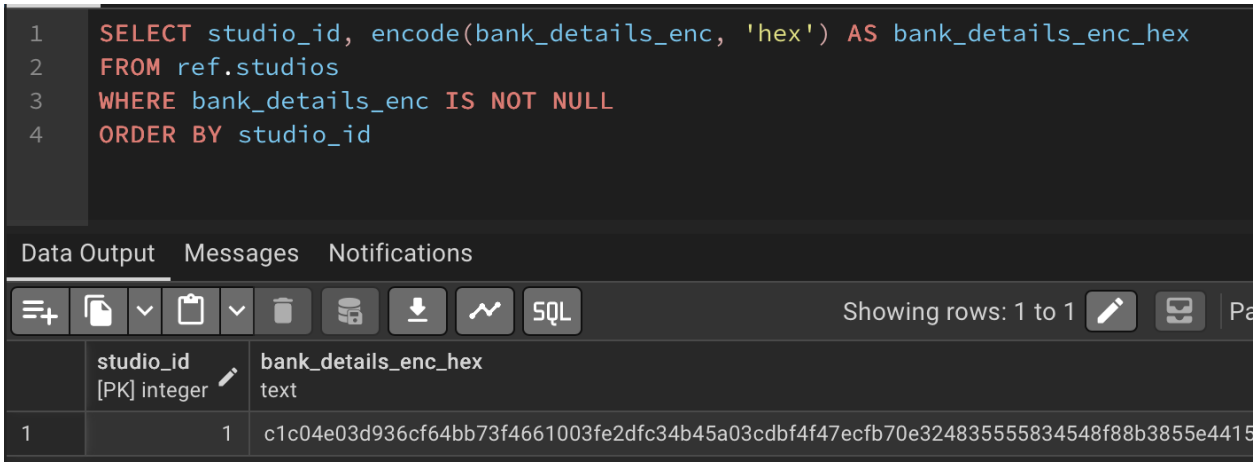


Рисунок 5 – Зашифрованные банковские данные

Далее проверим, что данные передаются в расшифрованном виде, передав ключ. На рисунке 6 расшифровываем с помощью закрытого ключа.

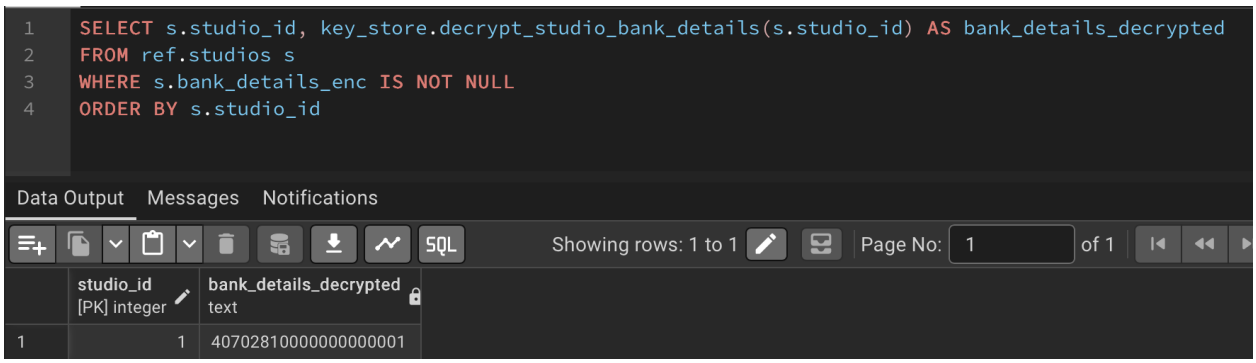


Рисунок 6 – Расшифрованные банковские данные с ключом

На рисунке 7 расшифровываем с помощью симметричного ключа.

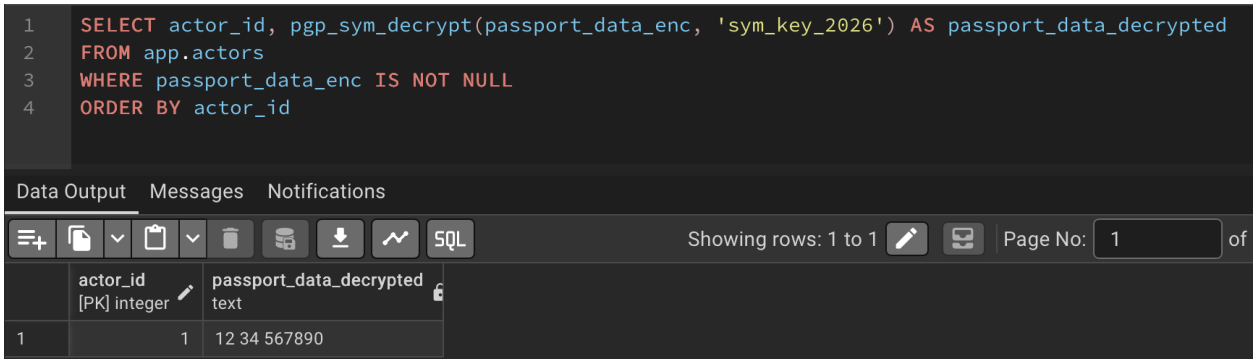


Рисунок 7 – Расшифрованные паспортные данные с ключом

Из рисунков 4-7 видим, что ключи успешно шифруют и расшифровывают нужные поля. Код реализации шифрования столбцов представлен в приложении Б.

3. Реализация безопасного метода хранения ключей шифрования

В нашей системе создана схема key_store с таблицей crypto_materials. В ней не хранятся сами ключи в открытом виде, а только записи о том, где каждый ключ находится. Доступ к схеме закрыт от обычных ролей и расшифровать может только security_admin (Рисунок 8).

	key_id [PK] bigint	key_name text	key_type text	key_value bytea	storage_location text
1	1	lr3_sym_key	metadata	[null]	docker-secrets:/var/lib/postgresql/secrets/lr3_sym_key.txt
2	2	lr3_pgp_public_key	metadata	[null]	docker-secrets:/var/lib/postgresql/secrets/lr3_pgp_public....
3	3	lr3_master_kek	metadata	[null]	docker-secrets:/var/lib/postgresql/secrets/lr3_master_kek....
4	4	lr3_pgp_private_key	metadata	[null]	docker-secrets:/var/lib/postgresql/secrets/lr3_pgp_private....
5	5	lr3_pgp_private_key_encrypt...	pgp_private_encrypt...	[binary da...	db:encrypted_blob; kek:docker-secrets/lr3_master_kek.txt

Рисунок 8 – Таблица key_store.crypto_materials

Ключ берётся не из таблицы, а через key_store.get_sym_key(), которая читает lr3_sym_key.txt с смонтированного тома, для ref.studios банковские реквизиты шифруются открытым PGP-ключом из файла lr3_pgp_public.asc.

В самих таблицах остаются только столбцы с шифротекстом и без ключей запрос просто показывает hex зашифрованных байт. Приложение и обычные пользователи БД работают с зашифрованными полями, ключи лежат вне базы данных на отдельном томе, а полный доступ к расшифровке только у security_admin через функции.

Также важно обеспечить аудит всех обращений к функциям чтения секретов: даже если содержимое ключа не попадает в логи, следует фиксировать факт вызова read_secret, время, инициатора и запрошенный файл. Параллельно с этим стоит настроить отдельное резервное копирование самих секретов, не смешивая их с дампами БД, и хранить резервные копии ключей в зашифрованном виде в ином, физически изолированном месте, периодически проверяя возможность восстановления. Можно заменить файловый том на специализированный менеджер секретов (например, HashiCorp Vault), который поддерживает динамическую выдачу временных ключей и автоматическую ротацию, а master_key должен находиться в аппаратном модуле безопасности (HSM), чтобы даже скомпрометированный хост не позволял извлечь мастер-ключ в открытом виде.

Код реализации безопасного хранения ключей шифрования представлен в приложении В.

4. Тесты производительности до и после включения шифрования

Проведем тесты производительности до и после включения шифрования для избранных столбцов с использованием EXPLAIN ANALYZE и оценим влияние шифрования на время выполнения запросов. На рисунке 11 представлен анализ запроса на выборку из таблицы без симметричного шифрования.

```
                                                                QUERY PLAN
------------------------------------------------------------------------------------------------------------
----------------------------
 Aggregate  (cost=423.25..423.26 rows=1 width=8) (actual time=1.818..1.818 rows=1 loops=1)
   Buffers: shared hit=148
   ->  Seq Scan on lr3_perf_sym_plain  (cost=0.00..398.00 rows=10101 width=0) (actual time=0.584
..1.449 rows=10000 loops=1)
         Filter: (passport_data ~~ 'PASSPORT-0001%'::text)
         Rows Removed by Filter: 10000
         Buffers: shared hit=148
 Planning:
   Buffers: shared hit=17
 Planning Time: 0.099 ms
 Execution Time: 1.830 ms
(10 rows)
```

Рисунок 11 – Анализ запроса до симметричного шифрования

На рисунке 12 представлен анализ запроса на выборку из таблицы с симметричным шифрованием.

```
                                                                QUERY PLAN
------------------------------------------------------------------------------------------------------------
----------------------------
 Aggregate  (cost=658.25..658.26 rows=1 width=8) (actual time=3388.783..3388.786 rows=1 loops=1)
   Buffers: shared hit=308
   ->  Seq Scan on lr3_perf_sym_enc  (cost=0.00..658.00 rows=100 width=0) (actual time=1699.704.
.3388.307 rows=10000 loops=1)
         Filter: (pgp_sym_decrypt(passport_data_enc, current_setting('app.lr3_sym_key'::text)) ~
~ 'PASSPORT-0001%'::text)
         Rows Removed by Filter: 10000
         Buffers: shared hit=308
 Planning:
   Buffers: shared hit=14
 Planning Time: 0.051 ms
 Execution Time: 3388.860 ms
(10 rows)
```

Рисунок 12 – Анализ запроса после симметричного шифрования

На рисунке 13 представлен анализ запроса на выборку из таблицы без ассиметричного шифрования.

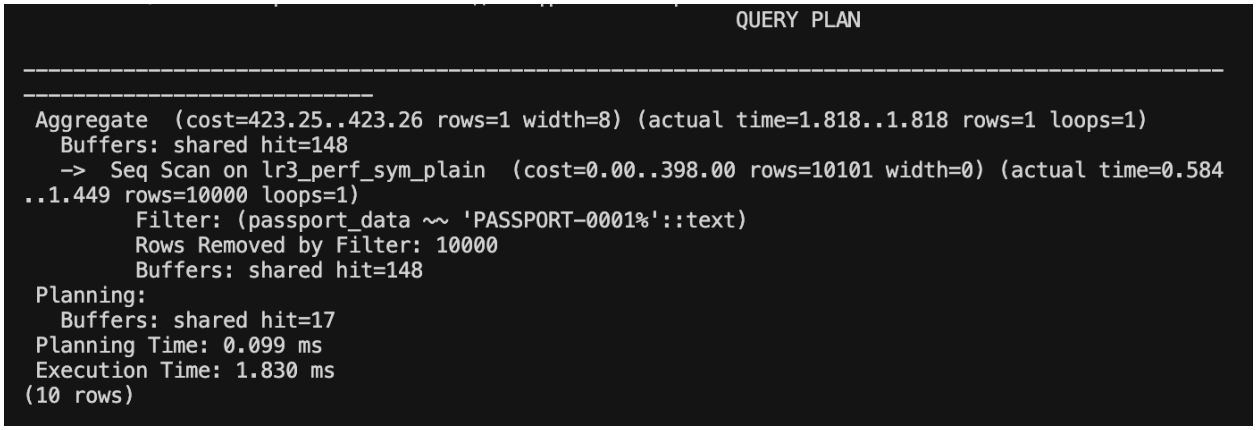


Рисунок 13 – Анализ запроса до ассиметричного шифрования

На рисунке 14 представлен анализ запроса на выборку из таблицы с ассиметричным шифрованием.

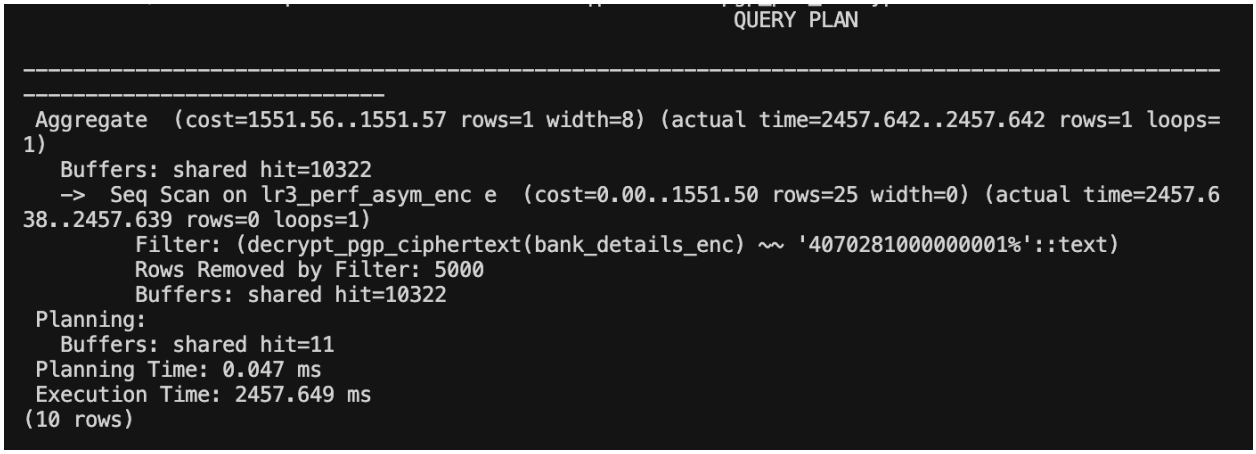


Рисунок 14 – Анализ запроса после асимметричного шифрования

Шифрование столбцов приводит к сильному увеличению времени чтения при условиях, где требуется дешифрование для фильтрации. До симметричного шифрования время выполнения 1,8 миллисекунды, после симметричного шифрования время выполнения выросло до 3388 миллисекунды, до асимметричного шифрования время выполнения 1,8 миллисекунды, после асимметричного шифрования время выполнения выросло до 2457 миллисекунды. Код тестов производительности представлен в приложении Г.

Вывод

В ходе выполнения лабораторной работы были изучены механизмы шифрования для защиты данных для предметной области студий дубляжа.

В первом задания был произведен анализ метода хеширования паролей учетных записей. Увидели, что используется устаревший алгоритм md5 и приняли решение изменить метод на scram-sha-256, обеспечивающий более высокий уровень защиты за счет использования соли и итераций.

Во втором задания выполнили шифрование конфиденциальных данных на уровне столбцов с использованием расширения pgcrypto. Для поля passport_data таблицы app.actors применили симметричное шифрование. Для поля bank_details таблицы ref.studios использовали асимметричное шифрование открытым PGP-ключом.

В третьем задании реализовали метод безопасного хранения ключей, при котором все ключи вынесены из базы данных в отдельный Docker-том. Создали схему key_store, где хранятся только метаданные о ключах и зашифрованный с помощью мастер-ключа закрытый PGP-ключ. Расшифровка данных происходит через функцию decrypt_studio_bank_details. Эта функция берет мастер-ключ из защищённого тома, расшифровывает закрытый PGP-ключ в памяти и дешифрует запрошенные данные, не раскрывая сам ключ вызывающей стороне. Доступ к функции ограничен security_admin, что невозможно использование расшифровки обычными пользователями.

В четвертом задании мы использовали тесты производительности с помощью EXPLAIN ANALYZE. После шифрования время выполнения выросло в 1882 раза, так как для фильтрации данных нужно последовательно расшифровывать каждую запись. После анализа выборки по асимметричному шифрованию время выполнения выросло в 1365 раз. Поэтому необходимо шифровать только критичные данные и использовать индексацию.

Приложение А

```
SET search_path TO public;

SELECT rolname, rolpassword
FROM pg_authid
WHERE rolname IN ('studio_manager', 'voice_actor', 'security_auditor',
'app_owner_user')
ORDER BY rolname;

-- Изменяем метод хеширования
ALTER SYSTEM SET password_encryption = 'scram-sha-256';
SELECT pg_reload_conf();

-- Изменяем пароли на новый метод
ALTER ROLE studio_manager WITH PASSWORD 'manager_pass123';
ALTER ROLE voice_actor WITH PASSWORD 'actor_pass123';
ALTER ROLE security_auditor WITH PASSWORD 'auditor_pass123';
ALTER ROLE app_owner_user WITH PASSWORD 'owner_pass123';

-- Проверка после изменения
SELECT rolname, rolpassword
FROM pg_authid
WHERE rolname IN ('studio_manager', 'voice_actor', 'security_auditor',
'app_owner_user')
ORDER BY rolname;
```

Приложение Б

```
SET search_path TO app, ref, key_store, public;

CREATE EXTENSION IF NOT EXISTS pgcrypto;

ALTER TABLE app.actors
   ADD COLUMN IF NOT EXISTS passport_data_enc BYTEA;

ALTER TABLE ref.studios
   ADD COLUMN IF NOT EXISTS bank_details_enc BYTEA;

DO $$
DECLARE
   v_sym_key TEXT;
   v_public_key TEXT;
BEGIN
   v_sym_key := key_store.get_sym_key();
   v_public_key := key_store.get_pgp_public_key();

   PERFORM set_config('app.lr3_sym_key', v_sym_key, false);
   PERFORM set_config('app.lr3_public_key', v_public_key, false);
END;
$$;

UPDATE app.actors
SET passport_data = '12 34 567890'
WHERE actor_id = (SELECT MIN(actor_id) FROM app.actors) AND
COALESCE(passport_data, '') = '';
```

```
UPDATE ref.studios
SET bank_details = '40702810000000000001'
WHERE studio_id = (SELECT MIN(studio_id) FROM ref.studios) AND
COALESCE(bank_details, '') = '';

UPDATE app.actors
SET passport_data_enc = pgp_sym_encrypt(passport_data,
current_setting('app.lr3_sym_key'))
WHERE passport_data IS NOT NULL AND (passport_data_enc IS NULL OR
passport_data_enc IS DISTINCT FROM pgp_sym_encrypt(passport_data,
current_setting('app.lr3_sym_key')));

UPDATE ref.studios
SET bank_details_enc = pgp_pub_encrypt(bank_details,
dearmor(current_setting('app.lr3_public_key')))
WHERE bank_details IS NOT NULL AND (bank_details_enc IS NULL OR
bank_details_enc IS DISTINCT FROM pgp_pub_encrypt(bank_details,
dearmor(current_setting('app.lr3_public_key'))));
```

Приложение В

```
CREATE SCHEMA IF NOT EXISTS key_store AUTHORIZATION app_owner;

SET search_path TO key_store, app, ref, public;

CREATE TABLE key_store.crypto_materials (
   key_id BIGSERIAL PRIMARY KEY,
   key_name TEXT NOT NULL UNIQUE,
   key_type TEXT NOT NULL CHECK (key_type IN ('pgp_private_encrypted',
'metadata')),
   key_value BYTEA,
   storage_location TEXT NOT NULL,
   created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
   created_by TEXT NOT NULL DEFAULT CURRENT_USER);

ALTER TABLE key_store.crypto_materials OWNER TO app_owner;

-- Доступ к схеме и таблице только для security_admin
REVOKE ALL ON SCHEMA key_store FROM PUBLIC;
REVOKE ALL ON ALL TABLES IN SCHEMA key_store FROM PUBLIC;
REVOKE ALL ON ALL SEQUENCES IN SCHEMA key_store FROM PUBLIC;
GRANT USAGE ON SCHEMA key_store TO security_admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON key_store.crypto_materials
TO security_admin;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA key_store TO
security_admin;
```

```
-- Функция для чтения содержимого файла-секрета из тома
CREATE OR REPLACE FUNCTION key_store.read_secret(p_filename TEXT)
RETURNS TEXT
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = key_store, public
AS $$
DECLARE
   v_path TEXT;
   v_content TEXT;
BEGIN
   -- Защита от выхода за пределы каталога
   IF p_filename !~ '^[a-z0-9_.-]+$' THEN
      RAISE EXCEPTION 'Недопустимое имя файла секрета: %', p_filename;
   END IF;
   v_path := '/var/lib/postgresql/secrets/' || p_filename;
   BEGIN
      v_content := pg_read_file(v_path);
   EXCEPTION
      WHEN OTHERS THEN
         RAISE EXCEPTION 'Секрет не найден: % (проверьте том ./secrets ->
/var/lib/postgresql/secrets). %',
            v_path, SQLERRM;
   END;
   IF length(trim(both E'\n\r' from v_content)) = 0 THEN
      RAISE EXCEPTION 'Секрет пуст: %', v_path;
   END IF;
   RETURN trim(both E'\n\r' from v_content);
END;
$$;
```

```
-- Функция для получение симметричного ключа
CREATE OR REPLACE FUNCTION key_store.get_sym_key()
RETURNS TEXT
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = key_store, public
AS $$
BEGIN
   -- Проверка прав
   IF SESSION_USER <> 'postgres'
      AND NOT pg_has_role(SESSION_USER, 'security_admin', 'member') THEN
       RAISE EXCEPTION 'Недостаточно прав: требуется postgres или
security_admin';
   END IF;
   RETURN key_store.read_secret('lr3_sym_key.txt');
END;
$$;

-- Функция для получение открытого PGP-ключа
CREATE OR REPLACE FUNCTION key_store.get_pgp_public_key()
RETURNS TEXT
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = key_store, public
AS $$
BEGIN
   RETURN key_store.read_secret('lr3_pgp_public.asc');
END;
$$;
```

```
-- Назначение владельца всем функциям чтения секретов
ALTER FUNCTION key_store.read_secret(TEXT) OWNER TO postgres;
ALTER FUNCTION key_store.get_sym_key() OWNER TO postgres;
ALTER FUNCTION key_store.get_pgp_public_key() OWNER TO postgres;

-- Отзыв публичного доступа к функциям и выдача привилегий
REVOKE ALL ON FUNCTION key_store.read_secret(TEXT) FROM PUBLIC;
REVOKE ALL ON FUNCTION key_store.get_sym_key() FROM PUBLIC;
REVOKE ALL ON FUNCTION key_store.get_pgp_public_key() FROM
PUBLIC;
GRANT EXECUTE ON FUNCTION key_store.read_secret(TEXT) TO
security_admin;
GRANT EXECUTE ON FUNCTION key_store.get_sym_key() TO
security_admin;
GRANT EXECUTE ON FUNCTION key_store.get_pgp_public_key() TO
security_admin, app_owner;

-- Добавление метаданных в таблицу
INSERT INTO key_store.crypto_materials (key_name, key_type, key_value,
storage_location)
VALUES
   ('lr3_sym_key', 'metadata', NULL, 'docker-
secrets:/var/lib/postgresql/secrets/lr3_sym_key.txt'),
   ('lr3_pgp_public_key', 'metadata', NULL, 'docker-
secrets:/var/lib/postgresql/secrets/lr3_pgp_public.asc'),
   ('lr3_master_kek', 'metadata', NULL, 'docker-
secrets:/var/lib/postgresql/secrets/lr3_master_kek.txt'),
   ('lr3_pgp_private_key', 'metadata', NULL, 'docker-
secrets:/var/lib/postgresql/secrets/lr3_pgp_private.asc')
ON CONFLICT (key_name) DO UPDATE
```

```
SET key_type = EXCLUDED.key_type,
    storage_location = EXCLUDED.storage_location,
    key_value = EXCLUDED.key_value;

-- Сохранение зашифрованного закрытого PGP-ключа
INSERT INTO key_store.crypto_materials (key_name, key_type, key_value,
storage_location)
VALUES (
    'lr3_pgp_private_key_encrypted',
    'pgp_private_encrypted',
    pgp_sym_encrypt(
        key_store.read_secret('lr3_pgp_private.asc'),
        key_store.read_secret('lr3_master_kek.txt')
    ),
    'db:encrypted_blob; kek:docker-secrets/lr3_master_kek.txt'
)
ON CONFLICT (key_name) DO UPDATE
SET key_type = EXCLUDED.key_type,
    key_value = EXCLUDED.key_value,
    storage_location = EXCLUDED.storage_location;

-- Функция для расшифровки банковских реквизитов
CREATE OR REPLACE FUNCTION
key_store.decrypt_studio_bank_details(p_studio_id INTEGER)
RETURNS TEXT
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = key_store, ref, public
AS $$
DECLARE
```

```
    v_private_key TEXT;
    v_private_key_encrypted BYTEA;
    v_bank_details_enc BYTEA;
    v_kek TEXT;
BEGIN
    IF SESSION_USER <> 'postgres'
       AND NOT pg_has_role(SESSION_USER, 'security_admin', 'member') THEN
        RAISE EXCEPTION 'Недостаточно прав для расшифровки: требуется
security_admin';
    END IF;

    -- Читаем зашифрованный закрытый ключ из таблицы
    SELECT key_value
      INTO v_private_key_encrypted
      FROM key_store.crypto_materials
     WHERE key_name = 'lr3_pgp_private_key_encrypted';

    IF v_private_key_encrypted IS NULL THEN
        RAISE EXCEPTION 'Зашифрованный закрытый ключ не найден в
key_store.crypto_materials';
    END IF;

    -- Извлекаем мастер ключ из файла и расшифровываем закрытый ключ
    v_kek := key_store.read_secret('lr3_master_kek.txt');
    v_private_key := pgp_sym_decrypt(v_private_key_encrypted, v_kek);

    -- Получаем зашифрованные банковские данные
    SELECT s.bank_details_enc
      INTO v_bank_details_enc
      FROM ref.studios s
```

```
    WHERE s.studio_id = p_studio_id;

   IF v_bank_details_enc IS NULL THEN
      RETURN NULL;
   END IF;

   -- Дешифровка и возврат открытых данных
   RETURN pgp_pub_decrypt(v_bank_details_enc, dearmor(v_private_key));
END;
$$;

-- Функция для расшифровки PGP-шифротекста
CREATE OR REPLACE FUNCTION
key_store.decrypt_pgp_ciphertext(p_ciphertext BYTEA)
RETURNS TEXT
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = key_store, public
AS $$
DECLARE
   v_private_key TEXT;
   v_private_key_encrypted BYTEA;
   v_kek TEXT;
BEGIN
   IF p_ciphertext IS NULL THEN
      RETURN NULL;
   END IF;

   IF SESSION_USER <> 'postgres'
     AND NOT pg_has_role(SESSION_USER, 'security_admin', 'member') THEN
```

```
        RAISE EXCEPTION 'Недостаточно прав для расшифровки: требуется
security_admin';
    END IF;

    -- Получаем зашифрованный закрытый ключ
    SELECT key_value
      INTO v_private_key_encrypted
      FROM key_store.crypto_materials
     WHERE key_name = 'lr3_pgp_private_key_encrypted';

    v_kek := key_store.read_secret('lr3_master_kek.txt');
    v_private_key := pgp_sym_decrypt(v_private_key_encrypted, v_kek);

    RETURN pgp_pub_decrypt(p_ciphertext, dearmor(v_private_key));
END;
$$;

-- Назначение владельца и настройка прав для функций расшифровки
ALTER FUNCTION key_store.decrypt_studio_bank_details(INTEGER) OWNER
TO postgres;
ALTER FUNCTION key_store.decrypt_pgp_ciphertext(BYTEA) OWNER TO
postgres;
REVOKE ALL ON FUNCTION
key_store.decrypt_studio_bank_details(INTEGER) FROM PUBLIC;
REVOKE ALL ON FUNCTION key_store.decrypt_pgp_ciphertext(BYTEA)
FROM PUBLIC;
GRANT EXECUTE ON FUNCTION
key_store.decrypt_studio_bank_details(INTEGER) TO security_admin;
GRANT EXECUTE ON FUNCTION key_store.decrypt_pgp_ciphertext(BYTEA)
TO security_admin;
```

Приложение Г

```
SET search_path TO app, key_store, public;

-- Таблицы для симметричного шифрования
CREATE TABLE app.lr3_perf_sym_plain (
   id BIGSERIAL PRIMARY KEY,
   passport_data TEXT NOT NULL
);

CREATE TABLE app.lr3_perf_sym_enc (
   id BIGSERIAL PRIMARY KEY,
   passport_data_enc BYTEA NOT NULL
);

INSERT INTO app.lr3_perf_sym_plain(passport_data)
SELECT 'PASSPORT-' || lpad(g::text, 8, '0')
FROM generate_series(1, 20000) AS s(g);

INSERT INTO app.lr3_perf_sym_enc(passport_data_enc)
SELECT pgp_sym_encrypt('PASSPORT-' || lpad(g::text, 8, '0'),
current_setting('app.lr3_sym_key'))
FROM generate_series(1, 20000) AS s(g);

-- До симметричного шифрования
EXPLAIN (ANALYZE, BUFFERS, TIMING)
SELECT COUNT(*)
FROM app.lr3_perf_sym_plain
WHERE passport_data LIKE 'PASSPORT-0001%';
```

```
-- После симметричного шифрования
EXPLAIN (ANALYZE, BUFFERS, TIMING)
SELECT COUNT(*)
FROM app.lr3_perf_sym_enc
WHERE pgp_sym_decrypt(passport_data_enc,
current_setting('app.lr3_sym_key')) LIKE 'PASSPORT-0001%';


-- Таблицы для асимметричного шифрования
CREATE TABLE app.lr3_perf_asym_plain (
   id BIGSERIAL PRIMARY KEY,
   bank_details TEXT NOT NULL
);

CREATE TABLE app.lr3_perf_asym_enc (
   id BIGSERIAL PRIMARY KEY,
   bank_details_enc BYTEA NOT NULL
);

INSERT INTO app.lr3_perf_asym_plain(bank_details)
SELECT '40702810' || lpad(g::text, 12, '0')
FROM generate_series(1, 5000) AS s(g);

INSERT INTO app.lr3_perf_asym_enc(bank_details_enc)
SELECT pgp_pub_encrypt('40702810' || lpad(g::text, 12, '0'),
dearmor(current_setting('app.lr3_public_key')))
FROM generate_series(1, 5000) AS s(g);

-- До асимметричного шифрования
EXPLAIN (ANALYZE, BUFFERS, TIMING)
```

```
SELECT COUNT(*)
FROM app.lr3_perf_asym_plain
WHERE bank_details LIKE '4070281000000001%';

-- После асимметричного шифрования
EXPLAIN (ANALYZE, BUFFERS, TIMING)
SELECT COUNT(*)
FROM app.lr3_perf_asym_enc e
WHERE key_store.decrypt_pgp_ciphertext(e.bank_details_enc) LIKE
'4070281000000001%';
```